



ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Knowledge Graph Embedding

Prof. Gianluca Moro
Tutor: Dr. Giacomo Frisoni

DISI, University of Bologna, Cesena
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{gianluca.moro, giacomo.frisoni}@unibo.it

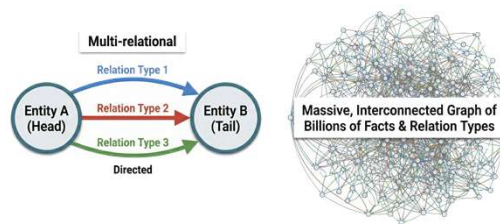


Outline

- **Introduction** 
- Anatomy of a knowledge graph embedding model
- Datasets
- Evaluation protocol and metrics
- Transactional distance models

Knowledge graph: definition

- A knowledge graph (KG) is a **directed**, **multi-relational** graph where the edges are not merely connectors, but carriers of specific **semantic** facts
- KGs are **triple stores** (unit = head-relation-tail, or subject-predicate-object)
 - $\mathcal{G} = \{(h, r, t)\} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ where $\mathcal{E}$ = entity vocabulary, $\mathcal{R}$ = relation vocabulary
 - E.g., (Paris, is_capital_of, France)
 - Compared to a general graph, **relations are first-class citizens** (focus on the connection's *content*)
- KGs in NLP and industry are frequently **massive**, containing billions of facts and thousands of different relation types within $\mathcal{R}$

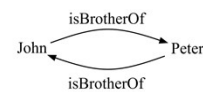


Knowledge graph: relational patterns

- Relations in a KG can generally obey specific logical rules

Symmetry

$$\forall e_1, e_2 : (e_1, r, e_2) \rightarrow (e_2, r, e_1)$$



Asymmetry (property of a single relation)

$$\forall e_1, e_2 : (e_1, r, e_2) \rightarrow \neg(e_2, r, e_1)$$

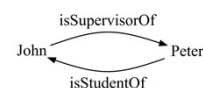
If a relation exists in one direction, it cannot exist in the reverse direction w/ same label



Inversion

$$\forall e_1, e_2 : (e_1, r_1, e_2) \rightarrow (e_2, r_2, e_1)$$

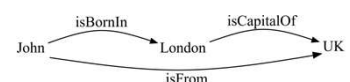
If a relation exists in one direction w/ r_1 , it must exist in the reverse direction w/ r_2



Composition

$$\forall e_1, e_2, e_3 : (e_1, r_1, e_2) \wedge (e_2, r_2, e_3) \rightarrow (e_1, r_k, e_3)$$

And more: hierarchies, type constraints, transitivity, homophily, ...

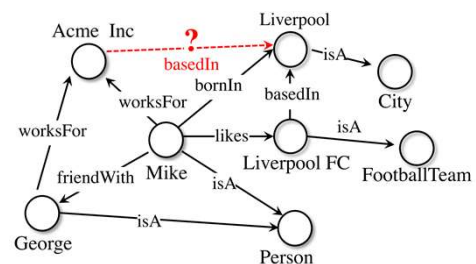


Knowledge graph: incompleteness

- **Closed World Assumption (CWA)** ← Used by general graphs
 - If a fact is not in the database, it is False
 - Adding new information acts as a revision of previous «truths»
- **Open World Assumption (OWA)** ← Used by KGs
 - The absence of a fact does not imply it is False; it is **unknown**
 - **KGs operate on the assumption they are always incomplete**; in other words, we list only true triplets and we accept the possibility that a true triplet is missing



FYK. To manage a **dynamic KG** effectively, OWA is often combined with **temporal scoping**. We move from triplets to **quadruples**: $(h, r, t) \rightarrow (h, r, t, \tau)$, where τ represents the validity time.

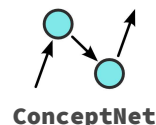


Knowledge graph: popular examples



The largest open-source, encyclopedic, collaboratively edited KG. It is the central storage for the structured data of Wikipedia. It is highly active and evolves every second.

Number of triples: ~1.65B
Number of relation types: ~13,000
Number of entities: ~119M

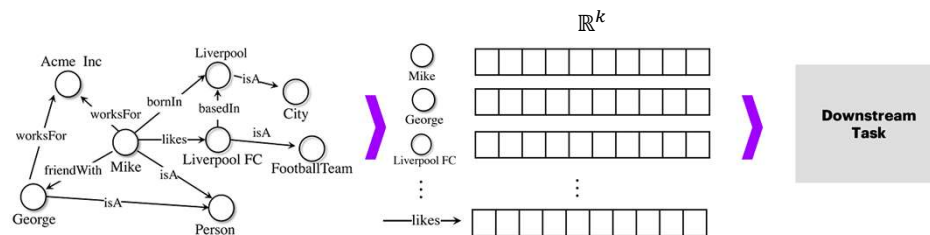


Semantic network designed to give computers «common sense». It stores the intuitive knowledge that humans use to understand language and daily life.

Number of triples: ~21M
Number of relation types: 36
Number of entities: ~8M

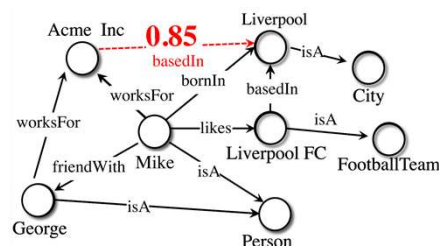
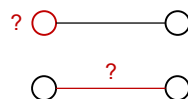
Knowledge graph embedding: definition

- The process of mapping the **symbolic components** of a KG into **continuous, low-dimensional vectors spaces** that preserve the logical structure
- A specialized subfamily of Graph Representation Learning
 - The relation embedding is just as important as the entity embedding
 - Relations have **intrinsic semantic properties (logic)** that the model must learn
 - Relations are not just connecting data; they are *transforming* data
 - E.g., the relation `is_father_of` might be learned as a specific vector translation that moves a point in space from "Generation X" coordinates to "Generation X+1" coordinates



Knowledge graph embedding: two popular tasks

- **Triple classification**
 - Assigning semantic labels or classes to triples (organizing knowledge)
 - Classification metrics
 - E.g., (Tom Hanks, starred_in, Forrest Gump) → Movie
- **Link prediction (focus of this lecture)**
 - Assigning a score proportional to the likelihood that an unseen triple is true (discovering new facts)
 - Information retrieval metrics (learning to rank problem)
 - It can be viewed as a binary classification task

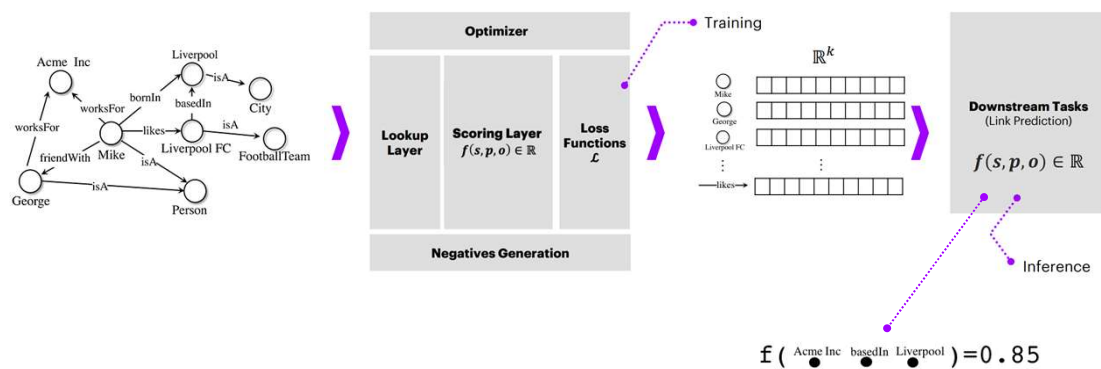


Outline

- Introduction
- **Anatomy of a knowledge graph embedding model** 
- Datasets
- Evaluation protocol and metrics
- Transactional distance models

At a glance

- Distinguish between **True (existing)** and **False (non-existing)** triples
 - Generate a set of negative samples that do not exist in the KG
 - Then, train a model to assign lower scores to negative samples compared to the positive ones



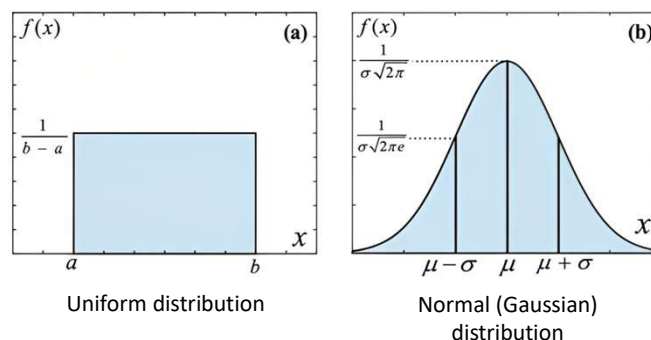
Building blocks

- 1 **Lookup layer**
 - Responsible for embedding nodes, edges, and their features
- 2 **Scoring function f** *• Since they characterize a KGE technique, we will see them later*
 - Given the embeddings, it computes a probability score for the triple depending on the specific link prediction algorithm being used
 - High score = High chance for a triple to be a true fact
- 3 **Loss function $\mathcal{L}$**
 - It quantifies the dissimilarity between the predicted scores and the actual labels (presence or absence of links) in the training data
- 4 **Optimizer**
 - It updates the model's parameters to minimize the loss function during the training process
 - E.g., Vanilla Gradient Descent and its variants, such as Adam or RMSprop
- 5 **Negative sampling strategy**
 - Crucial to create a balanced training dataset

Entity and relation initializers: random

1

- **Random**
 - Random vectors sampled from a uniform or Gaussian distribution
 - + Simpler and popular method
 - - It may require longer training times to achieve meaningful representations



Entity and relation initializers: label and structure 1

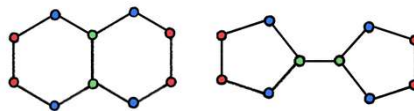
• Label-based

- Pretrained models from the HuggingFace `transformers` library to encode labels
- E.g., 768-dim embeddings from BERT (+ leverage dense semantic knowledge)



• Weisfeiler-Lehman (WL)

- The WL algorithm captures local neighborhood information and structural patterns in the graph by assigning colors to nodes and refining them iteratively
- Two nodes with the same color after the same number of iterations in the WL algorithm have the same local structural information in their neighborhoods
- Categorical colors obtained during the WL algorithm's execution are transformed into embeddings
- Initial representations reflecting the graph's local structure can help the model understand and reason about the KG better, improving convergence and generalization



Entity and relation initializers: position (1/2) 1

• Random-Walk Positional Encoding (RWPE) [Dwivedi et al., 2022]

- Positional encoding adds information about the relative position of entities or relations in the graph
- Random walks are stochastic processes where, at each step, a walker moves to one of its neighboring nodes with a certain probability
- RWPE initializes nodes with a d -dim vector encoding the landing probabilities of a node i to itself in 1 to d steps of random walk (precomputed before the model training)
- + Help in distinguishing (i) structurally dissimilar nodes, (ii) non-isomorphic graph patterns

$$x_i = [\mathbf{R}_{i,i}, \mathbf{R}_{i,i}^2, \dots, \mathbf{R}_{i,i}^d] \in \mathbb{R}^d$$

$\mathbf{A}$ = adjacency matrix

$\mathbf{D}$ = diagonal degree matrix = $\sum_i \mathbf{A}_{i,i}$

$\mathbf{R}$ = random walk matrix = $\mathbf{A}\mathbf{D}^{-1}$

← $\mathbf{A}_{ij} = 1$ if there is an edge between node i and node j , 0 otherwise

← $\mathbf{D}_{ii}$ = node i degree, i.e., number of edges incident to node i

← $\mathbf{D}^{-1}$ ensures normalization, i.e., the rows of $\mathbf{R}$ sum to 1

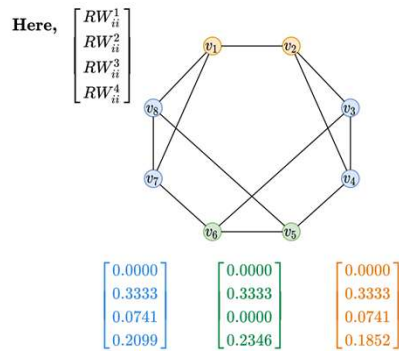
$\mathbf{R}$ models transition probabilities

Entity and relation initializers: position (2/2)

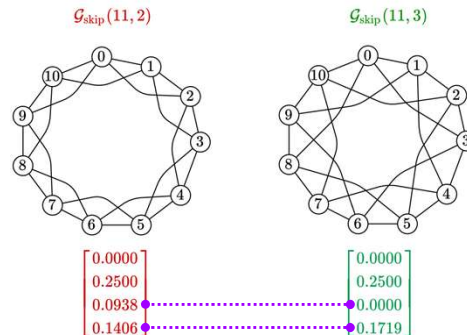
1

• Random-Walk Positional Encoding (RWPE) [Dwivedi et al., 2022]

- A graphical example



Structurally equal nodes have the same initial embedding



Each node in a graph has the same embedding.
Non-isomorphic graphs have different embeddings.

Common loss functions

3

• Pairwise Margin-Based Hinge Loss [Bordes et al., 2013]

- Push the score of the True triple to exceed the False triple by at least a margin γ
- Ranking goal

$$\mathcal{L}(\theta) = \sum_{t^+ \in \mathcal{G}} \sum_{t^- \in \mathcal{C}} \max(0, [\gamma + f(t^-; \theta) - f(t^+; \theta)])$$

• Cross-Entropy / Negative Log-Likelihood [Trouillon et al., 2016]

- Push the model to assign 100% probabilities to True triples and 0% to the False triples
- Truth goal

$$\mathcal{L}(\theta) = \sum_{t \in \mathcal{GUC}} \log(1 + e^{-y \cdot f(t; \theta)})$$

Gold label of the triple t

- +1 for True triples (from $\mathcal{G}$)
- -1 for False triples (from $\mathcal{C}$, i.e., corrupted set)

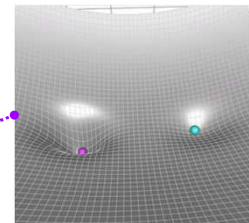
Optimizer recap: VGD and momentum

4

- $\Delta_{i,t}$ = amount of change for each parameter i at iteration t ; i.e. $\theta_{i+1} = \theta_i + \Delta_{i,t}$
- **Vanilla Gradient Descent**
 - Iterative steps in the opposite directions of gradients
 - $\Delta_{i,t} = -\eta * \frac{\partial \mathcal{L}}{\partial \theta_i} \leftarrow \eta = \text{learning rate}, \frac{\partial \mathcal{L}}{\partial \theta_i} \text{ or } \nabla \theta_i = \text{gradient}$
- **Momentum**
 - It accumulates past gradients to move faster and have a shot of escaping local minima
 - $\Delta_{i,t} = -\eta * \frac{\partial \mathcal{L}}{\partial \theta_i} + \gamma * \Delta_{i,t-1} \leftarrow \gamma = \text{decay rate}$
 - $\gamma = 0 \rightarrow$ the same as Vanilla Gradient Descent (full friction)
 - Typically, $\gamma \in [0.8, 0.9]$ (i.e., surface with little friction)

Momentum vs. **Vanilla Gradient Descent** on a surface with a global minimum (the left well) and local minimum (the right well)

- $\mathbf{x}, \mathbf{y}$ = parameters to optimize
- **Surface (z)** = loss function



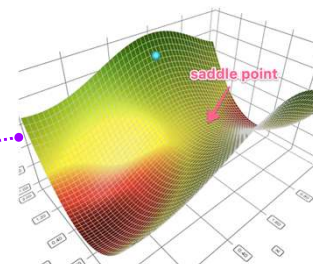
Optimizer recap: AdaGrad

4

- **Adaptive Gradient (AdaGrad)**
 - It adapts η for each parameter based on the historical gradients: η is scaled inversely proportional to the cumulative sum of squared gradients for that parameter
 - ☞ Give larger updates to parameters with smaller gradients and vice versa; in visual terms, how much you have updated a parameter is to say how much you have moved in that dimension
 - Useful with sparse features or settings where different parameters have significantly different scales
 - However, η keeps decreasing over time, leading to very small updates in later stages of training
 - $g_0 = 0$
 - $g_t = g_{t-1} + \left(\frac{\partial \mathcal{L}}{\partial \theta_i}\right)^2$
 - $\Delta_{i,t} = -\frac{\eta}{\sqrt{g_t + \epsilon}} * \frac{\partial \mathcal{L}}{\partial \theta_i}$

ϵ = small constant value (e.g., 1e-5 or 1e-8) to avoid division by zero

AdaGrad vs. **Vanilla Gradient Descent** on a terrain with a saddle point. AdaGrad's path is straighter regardless of η . Sometimes, vanilla gradient descent might just stop at the saddle point where gradients in both directions are 0 and be perfectly content there.



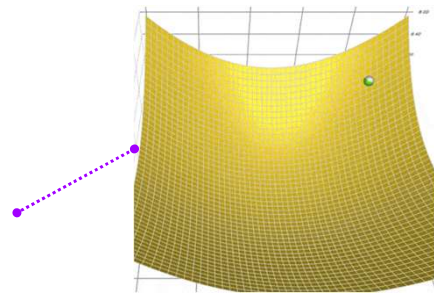
Optimizer recap: RMSProp

4

• Root Mean Square Propagation (RMSProp)

- Since the sum of gradient squared only grows and never shrinks, AdaGrad is slow
- RMSProp fixes this issue by adding a **decay factor** β to consider only recent gradients (the ones from long ago are basically forgotten)
- Differently from the decay rate in *Momentum*, the β here also has a scaling effect (e.g., if $\beta = 0.99$, the sum of gradient squared will be $\sqrt{1 - 0.99} = 0.1$ that of AdaGrad, i.e., 10x larger step with same η)
- $g_0 = 0$
- $g_t = \beta * g_{t-1} + (1 - \beta) * \left(\frac{\partial \mathcal{L}}{\partial \theta_i}\right)^2$
- $\Delta_{i,t} = -\frac{\eta}{\sqrt{g_t + \epsilon}} * \frac{\partial \mathcal{L}}{\partial \theta_i}$

RMSProp vs. **AdaGrad**. The first run just shows the balls; the second run also shows the sum of gradient squared represented by the squares.



Optimizer recap: Adam

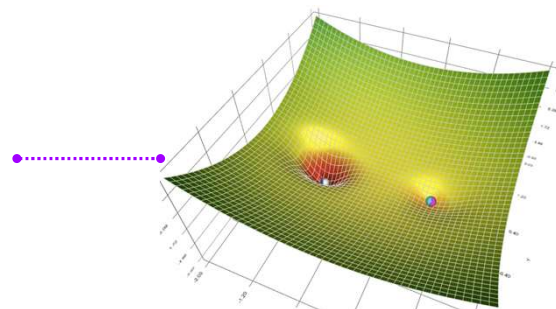
4

• Adaptive Moment Estimation (Adam)

- Takes the best of both worlds of *Momentum* and *RMSProp*: the go-to choice of Deep Learning
- Gets the speed from *Momentum* and the ability to adapt gradients in distinct directions from *RMSProp*
- $v_t = \beta_1 * v_{t-1} + (1 - \beta_1) * \left(\frac{\partial \mathcal{L}}{\partial \theta_i}\right)$ [Momentum] $\beta_1 = 1^\circ$ moment decay rate (e.g., 0.9)
- $s_t = \beta_2 * s_{t-1} + (1 - \beta_2) * \left(\frac{\partial \mathcal{L}}{\partial \theta_i}\right)^2$ [RMSProp] $\beta_2 = 2^\circ$ moment decay rate (e.g., 0.999)
- $\Delta_{i,t} = -\eta \frac{v_t}{\sqrt{s_t + \epsilon}} * \frac{\partial \mathcal{L}}{\partial \theta_i}$

Adam vs. **Vanilla Gradient Descent** vs. **Momentum** vs. **RMSProp** vs. **AdaGrad**. Left well is the global minimum; right well is a local minimum. Adam is favored due to its better performance and faster convergence.

[Play around yourself with other races](#)



Negatives generation: OWA reality vs LCWA necessity

5

• Where do negative examples come from?

- **Local Closed World Assumption (LCWA)**: the KG is only *locally* complete (meaning that it contains a limited set of facts that are known to be true, and any missing facts are assumed to be false)
- Synthetic negative examples can be created by randomly sampling entities and relations

$$\mathcal{C}(s, p, o) = \{(\hat{s}, p, o) | \hat{s} \in \mathcal{E} \wedge \hat{s} \neq s\} \cup \{(s, p, \hat{o}) | \hat{o} \in \mathcal{E} \wedge \hat{o} \neq o\}$$

«corrupted» subject

the predicate is typically unaltered

«corrupted» object

• Is LCWA a too strong assumption?

- **OWA is for inference, LCWA is for training**: we cannot learn from positive data alone; a necessary shortcut to prevent the model from collapsing into predicting True for everything (over-generalization)
- In reality, KGs are often incomplete, and missing facts do not necessarily imply falseness
- We train to distinguish between true and false triples based on the *available information* in the KG

Negatives generation: visualizing the corruption process

5

• Notation

- Set of potential negatives over all positive triples t^+ in $\mathcal{G}$

$$\mathcal{C} = \bigcup_{t^+ \in \mathcal{G}} \mathcal{C}(t^+)$$

• Example

- $\mathcal{E} = \{\text{Mike, Liverpool, AcmeInc, George, LiverpoolFC}\}$
- $\mathcal{R} = \{\text{bornIn, friendWith}\}$
- $t^+ = (\text{Mike bornIn LiverpoolFC})$



$\mathcal{C}(t^+)$	Mike	bornIn	AcmeInc
	Mike	bornIn	LiverpoolFC
	George	bornIn	LiverpoolFC
	AcmeInc	bornIn	LiverpoolFC

5

Negatives generation: basic strategies

• Complete set

- No sampling; use all possible synthetic negatives for each positive p
- It ensures that negative examples cover the entire space of facts that could exist in the KG but are absent in the positive examples
- It can be computationally expensive for large KGs (mind scalability)

• Uniform sampling

- Generate all possible synthetic negatives and sample n negatives for each positive triple t
- Each negative sample has an equal chance of being selected (i.e., the probability of any pair of entities not having a link is equal)

• Bernoulli sampling [\[Wang et al., 2014\]](#)

- Set different probabilities for replacing the subject or object when corrupting the triplet, which depends on the mapping property of the relation, i.e., 1-1, N-1, N-N
- Give more chance to replace s if the relation is 1-N or to replace o if the relation is N-1
- Probability for s replacement = $\frac{ops}{ops+spo}$; probability for o replacement = $\frac{sps}{ops+spo}$
- ops = average number of object entities per subject entity (sps = subject entities per object entity)

5

Negatives generation: advanced strategies

• Pseudo-typed sampling

- When corrupting subjects for triple (s, p, o) , it considers only the entities which co-occur as subjects in other triples having the predicate p ; the same for objects

• Degree-based sampling

- «Nodes with high degree are more likely to be connected»
- It assigns higher probabilities of selecting nodes with lower degrees as negatives

• Structural sampling

- It considers local or global structural properties (e.g., nodes sharing neighbors) to generate negative samples that are more contextually relevant

 Many more task-dependent strategies

Negatives generation: what if a False fact is True ?

5

! Warning

It should be taken into account that also a corrupted triple that is not part of the KG can represent a true fact (i.e., **false negative**). However, $|\mathcal{C}| \gg |\mathcal{G}|$, so the likelihood of generating a false negative is rather low. This general observation might not hold for all entities; e.g., for a hub entity which is connected to many other entities, there is a higher risk of encountering false negatives. Filtering could be necessary.

- **No Filtering**
 - Given their rarity, the filtering step is omitted to reduce computational cost
- **Filtering** [[Bordes et al., 2013](#)]
 - Remove from $\mathcal{C}$ all the triples that appear either in the training, validation, or test set
 - If the graph is huge, exact filtering could be prohibitive; [Bloom filters](#) are efficient approximations with a configurable desired error rate, based on hashes and frequently used in literature

Outline

- Introduction
- Anatomy of a knowledge graph embedding model
- **Datasets** 
- Evaluation protocol and metrics
- Transactional distance models

Inter-split constraints

- Datasets contain gold triples only (negative sampling is applied during training)
- Gold triple partitioning is random but not without constraints: there are common sense principles to respect
- **No Repeated Triples**
 - Each triple is exactly in one split ← the Test Set contains unseen (missing) triples
- **Training Set = Entity Coverage**
 - All entities should occur at least once in training triples ← it allows the model to learn meaningful representations for all entities and capture their latent relationships effectively
- Note: a Validation Set is required for early stopping
- 80/10/10 is the most popular splitting

Reciprocal and inverse triples

- **Make the model invariant to directionality arbitrary choice**
 - Injecting reciprocal or inverse triples in the Training Set helps the model in capturing the nature of *symmetric* or *inverted* relationships [Lacroix et al., 2018]
 - This property is also known as *semantic invariance* [Bailly et al., 2015]

Symmetric Relationship (Bidirectional)
Reciprocal Triple

<Jack isMarriedWith Althea>
<Althea isMarriedWith Jack>

Inverted Relationship (Flipped Direction)
Inverse Triple

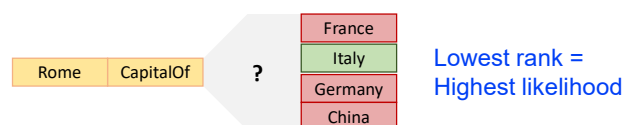
<Agatha childOf Jack>
<Jack childOf⁻¹ Agatha>

Outline

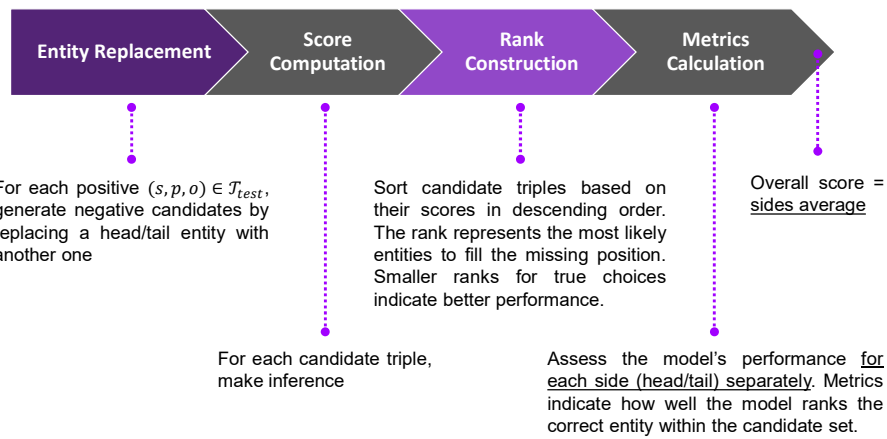
- Introduction
- Anatomy of a knowledge graph embedding model
- Datasets
- **Evaluation protocol and metrics** 
- Transactional distance models

Link prediction: a matter of placement

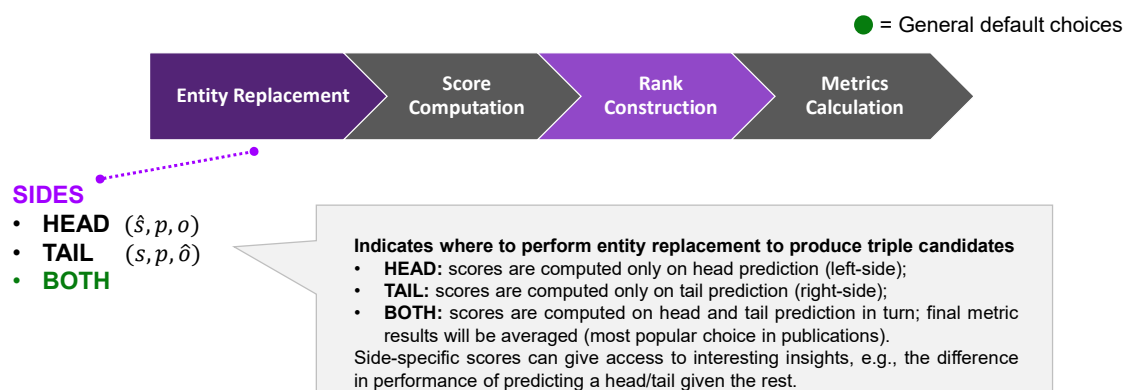
- **Training Recap**
 - A link prediction model is trained to assign a score or probability to each positive and negative example, indicating the likelihood of the triple being True
 - The model's objective is to discriminate positive and negative triples based on patterns, embeddings, or other representations
- **Evaluation**
 - Can the model accurately predict the missing triples not seen during training (Test Set) ?
- **Learning-to-Rank**
 - For each gold test triple, we generate negatives (same procedure used in training) by replacing the head and/or the tail with entities from the dictionary
 - How well are positive triples ranked against synthetic negatives built under the LCWA?



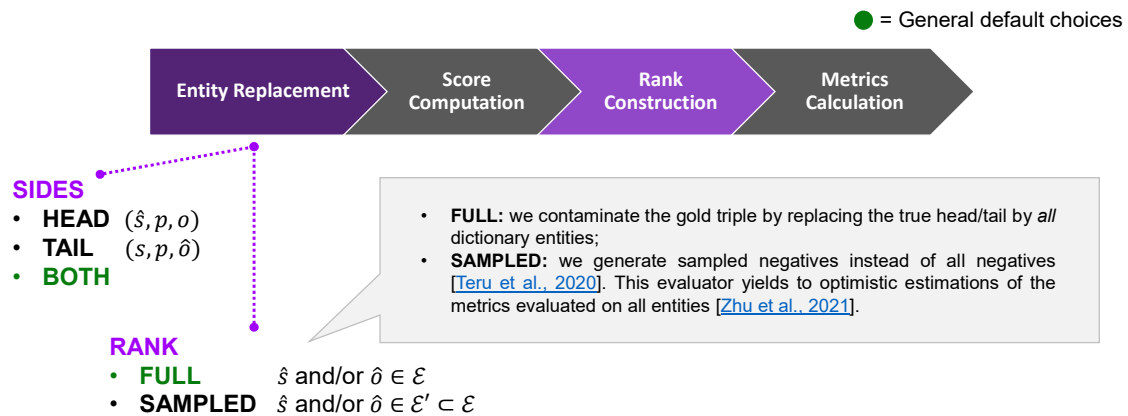
Configurable evaluation pipeline: overview



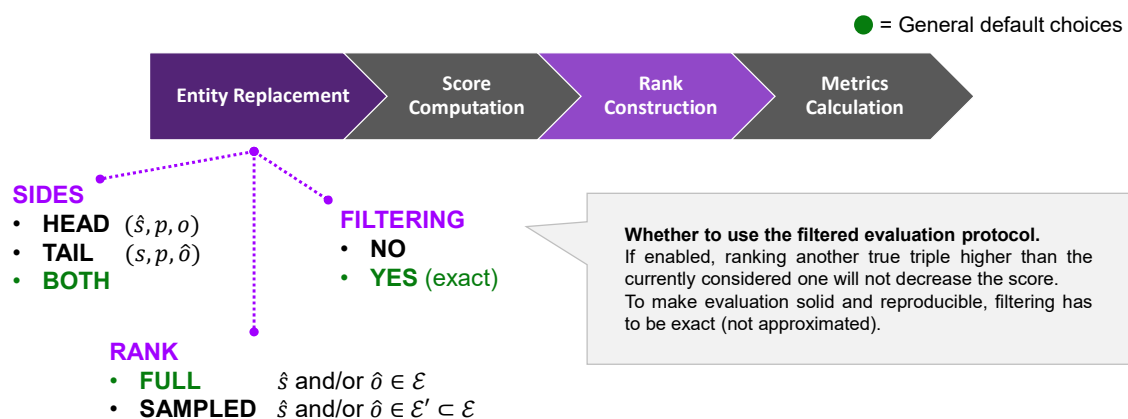
Configurable evaluation pipeline: entity replacement (1/3)



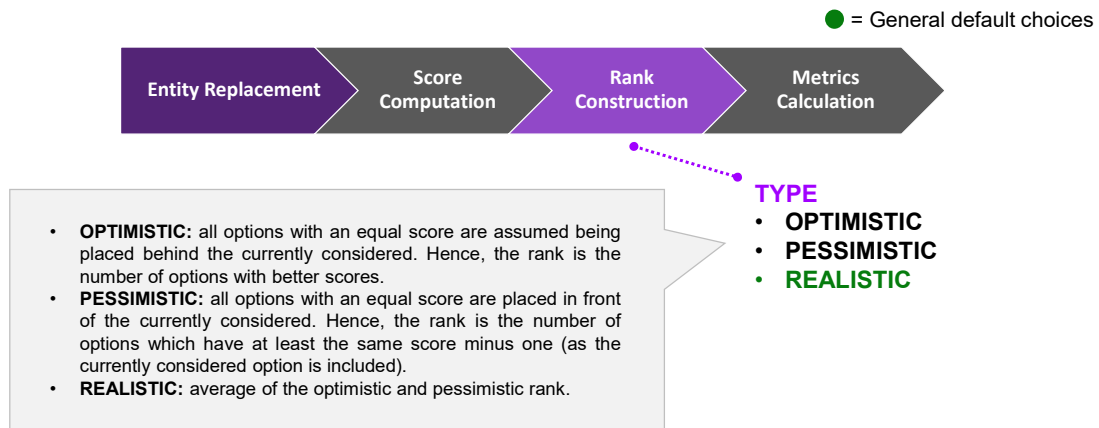
Configurable evaluation pipeline: entity replacement (2/3)



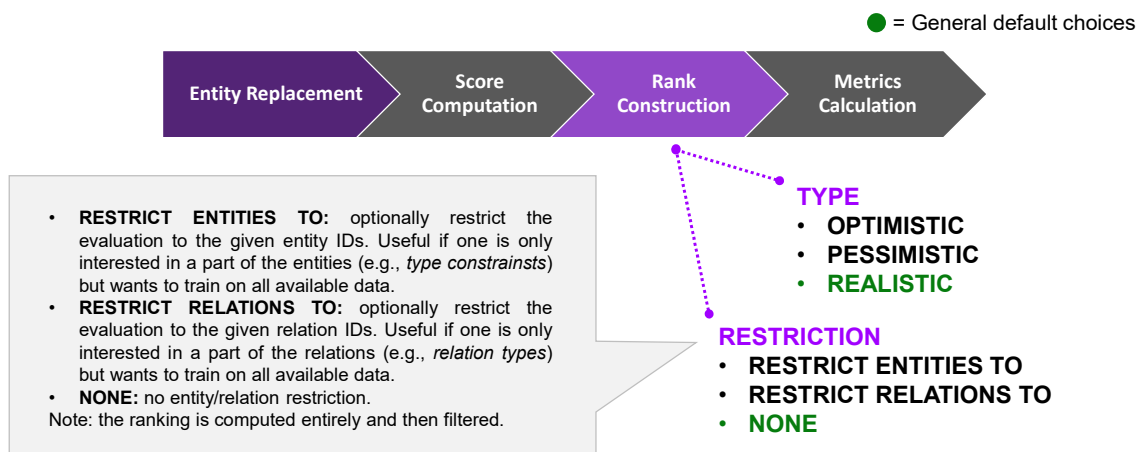
Configurable evaluation pipeline: entity replacement (3/3)



Configurable evaluation pipeline: rank construction (1/2)



Configurable evaluation pipeline: rank construction (2/2)



Automatic metrics: put True triples at the top

- Q = prediction ranks

- **Mean Rank (MR)**, $[1, |\mathcal{E}|]$ (for head or tail replacement), ↓

- Average rank of True triple completions

$$MR = \frac{1}{|Q|} \sum_{q \in Q} q$$

- **Mean Reciprocal Rank (MRR)**, $[0, 1]$, ↑

- Average reciprocal rank of True triple completions
- #1 → 1.0, #2 → 0.5, #10 → 0.1, 100 → 0.01
- Preferable to MR because of its greater robustness to outliers

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{q}$$

- **Hits@N**, $[0, 1]$, ↑

- How many of the correct missing links are in the first top- N ranks
- Just care about presence and not ordering
- $\mathbb{1}[q \leq N]$ yields 1 if q takes values between 1 and N , 0 otherwise
- Typically, $N \in \{1, 3, 10\}$

$$Hits@N = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[q \leq N]$$

Outline

- Introduction
- Anatomy of a knowledge graph embedding model
- Datasets
- Evaluation protocol and metrics
- Model taxonomy
- **Transactional distance models** 

TransE (2013)

Idea: Treats relations as simple translation vectors in the same embedding space—head plus relation should approximate tail.

Advantage: Simple, fast, and interpretable; good for modeling 1-to-1 relations.

TransH (2014)

Idea: Projects entities onto a relation-specific hyperplane before applying translation.

Advantage: Allows entities to play different roles in different relations; better handles N-to-1, 1-to-N relations.

TransR (2015)

Idea: Uses separate spaces for entities and relations, with relation-specific projection matrices.

Advantage: More expressive modeling of complex relational patterns; better captures relational semantics.

TransD (2015)

Idea: Creates dynamic projection matrices that depend on both relation and entity.

Advantage: More fine-grained modeling without over-parameterization; better for heterogeneous graphs.

TransSparse (2016)

Idea: Uses adaptive sparse projection matrices where sparsity depends on relation complexity.

Advantage: Prevents overfitting on simple relations and underfitting on complex ones; more parameter-efficient.

RotatE (2019)

Idea: Represents relations as rotations in complex space—head rotated by relation equals tail.

Advantage: Naturally models symmetric, inverse, and compositional relations; strong theoretical foundation.

QuatE (2019)

Idea: Extends RotatE using quaternions for 3D rotations.

Advantage: Captures richer relational interactions and latent dependencies; improves expressiveness.

HAKE (2020)

Idea: Models hierarchies by combining modulus and phase in polar coordinates.

Advantage: Explicitly captures hierarchical structures in knowledge graphs.

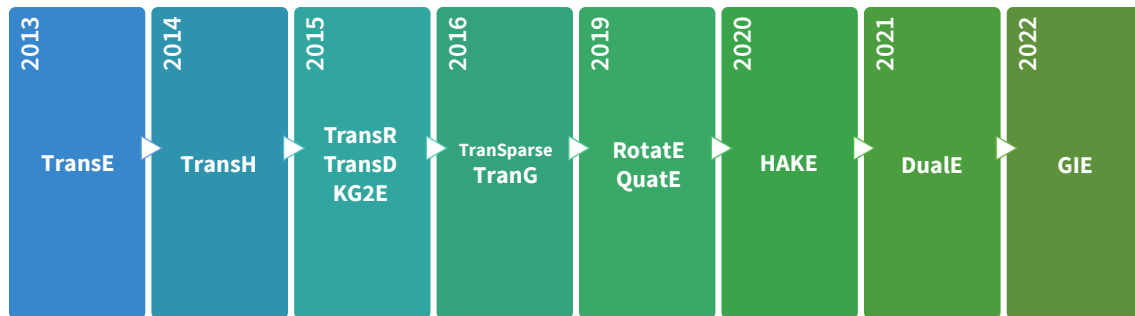
DualE & GIE

Idea: Operate in dual quaternion or geometric algebra spaces for non-Euclidean transformations.

Advantage: Suitable for modeling complex, non-linear relational patterns in high-dimensional spaces.

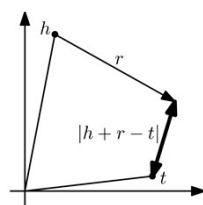
Transactional distance models

- Distance-based scoring function
 - Finding valid triples = translations of entities through relations
 - Termed **additive** as the vectors interact via *addition* and *subtraction* during training
 - Contributions are generally simple and benefit from solid interpretability



TransE [Bordes et al., 2013] : method

- The first and most classic translational model
- Entities and relations are represented as vectors of the same Euclidean space
 - Relations are interpreted as **translation vectors** connecting heads and tails with low error
- Margin-based ranking loss



$$\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^d$$

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$$

$$f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_{l_1/l_2}$$

Norm constraints
(l_1 by default)

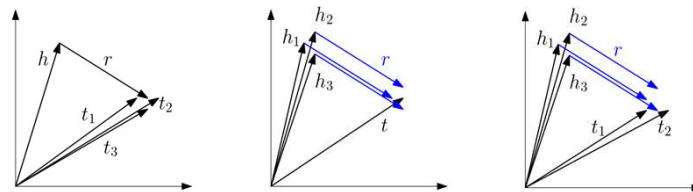
$$\|x\|_1 = \sum_1^n |x_i| \quad \|x\|_2 = \sqrt{\sum_1^n |x_i|^2}$$

Usually,
 $d \in [50, 300]$

The intuition comes from regularities observed in linguistic embeddings, word2vec [Mikolov et al., 2013]
E.g., king + woman $\approx$ queen + man

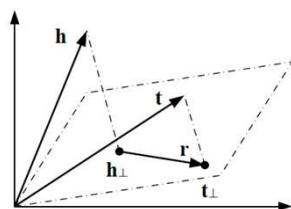
TransE [Bordes et al., 2013] : considerations

- + Simplicity, computationally efficient
- It inherently **cannot model 1-N, N-1, and N-N relationships**
 - For triples $(h, r, t_1), (h, r, t_2) \in \mathcal{G}$ where $t_1 \neq t_2$, the model adapts the embeddings in order to ensure that $\mathbf{h} + \mathbf{r} \approx \mathbf{t}_1$ and $\mathbf{h} + \mathbf{r} \approx \mathbf{t}_2$ which results in $\mathbf{t}_1 \approx \mathbf{t}_2$; the same holds for different heads
- It **fails in modeling symmetric relationships**



TransH [Wang et al., 2014]

- Translations on **relation-specific hyperplanes**
 - Entities are allowed to have distinct representations (roles) when involved in different relations
 - A hyperplane, represented by a normal vector $\mathbf{w}_r \in \mathbb{R}^d$, is learned for each relation
 - To compute the plausibility of a triple (h, r, t) , entities are projected on the hyperplane of r
 - The relation embedding $\mathbf{r}$ is used to translate all the entities within the same hyperplane
- Margin-based ranking loss



$$\begin{aligned} \mathbf{h}, \mathbf{r}, \mathbf{t} &\in \mathbb{R}^d \\ \mathbf{h}_\perp &= \mathbf{h} - \mathbf{w}_r^T \mathbf{h} \mathbf{w}_r \\ \mathbf{t}_\perp &= \mathbf{t} - \mathbf{w}_r^T \mathbf{t} \mathbf{w}_r \end{aligned}$$

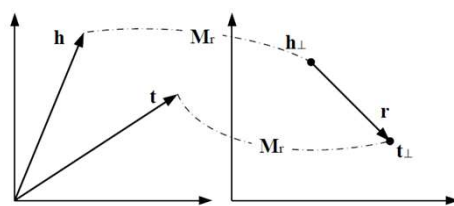
Usually,
 $d \in [50, 300]$

$$f_r(h, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2$$

Distance between the translated $\mathbf{h}$ and $\mathbf{t}$ in the embedding space under the relation-specific hyperplane

TransR [Lin et al., 2015]

- TransR shares a very similar idea with TransH but introduces **relation-specific spaces** (rather than hyperplanes) ← **TransH extension**
- Entities and relations are seen as different objects in different spaces
 - Entities are represented as vectors in an entity space $\mathbb{R}^d$
 - Each relation is associated with a specific space $\mathbb{R}^k$ and modeled as a translation vector in that space
 - Given a triple (h, r, t) , the entity embeddings $\mathbf{h}$ and $\mathbf{t}$ are first projected into the relation space by means of a relation-specific projection matrix $\mathbf{M}_r \in \mathbb{R}^{k \times d}$
- Margin-based ranking loss



$$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$$

$$\mathbf{r} \in \mathbb{R}^k$$

$$\mathbf{h}_\perp = \mathbf{M}_r \mathbf{h}$$

$$\mathbf{t}_\perp = \mathbf{M}_r \mathbf{t}$$

Usually, $d = 50, k = 300$

$$f_r(h, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2$$

– It requires $\theta(d \cdot k)$ parameters per relation, thereby losing the simplicity and efficiency of TransE/TransH

TransD [Ji et al., 2015]

- Like TransR, TransD considers entities and relations as objects living in different vector spaces...
 - ... However, instead of performing the same relation-specific projection for all entity embeddings, **entity-relation-specific projection matrices** $\mathbf{M}_{r,h}, \mathbf{M}_{r,t} \in \mathbb{R}^{k \times d}$ are constructed ← **TransR extension**
- Two separate embedding vectors for representing each entity and relation
 1. The first set of embeddings represents the semantics of entities and relations $\mathbf{h}, \mathbf{t} \in \mathbb{R}^d, \mathbf{r} \in \mathbb{R}^k$
 2. The second set of embeddings is used for calculating $\mathbf{M}_{r,h}, \mathbf{M}_{r,t}$ (dynamic projection matrices)
- Margin-based ranking loss

$$\mathbf{h}, \hat{\mathbf{h}}, \mathbf{t}, \hat{\mathbf{t}} \in \mathbb{R}^d$$

$$\mathbf{r}, \hat{\mathbf{r}} \in \mathbb{R}^k$$

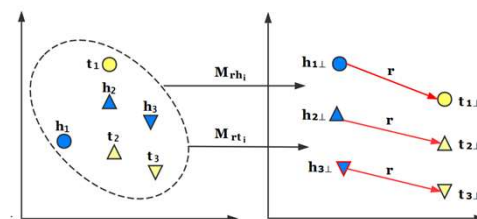
$$f_r(h, t) = -\|\mathbf{M}_{r,h} \mathbf{h} + \mathbf{r} - \mathbf{M}_{r,t} \mathbf{t}\|_2^2$$

$$\mathbf{M}_{r,h} = \hat{\mathbf{h}} \hat{\mathbf{h}}^T + \tilde{\mathbf{I}}$$

$$\mathbf{M}_{r,t} = \hat{\mathbf{t}} \hat{\mathbf{t}}^T + \tilde{\mathbf{I}}$$

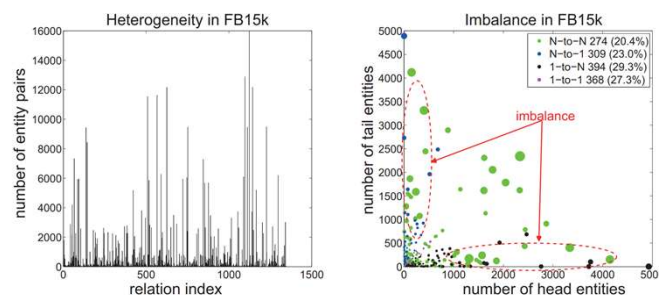
$\tilde{\mathbf{I}} \in \mathbb{R}^{k \times d}$ is a matrix with 1 on the diagonal and 0 elsewhere

Usually, $d = k = 50$



TranSparse [Ji et al., 2016] : motivations

- TransE/H/R/D ignore...
 - **Heterogeneity**: some relations link many entity pairs and others do not
 - it may lead to overfitting on simple relations or underfitting on complex relations
 - **Imbalance**: some relations link many head entities and fewer tail entities, and viceversa, e.g., the relation *gender* can link many person names at head and only male/female at tail;
 - imbalance of the two sides indicates that it is unreasonable to treat them equally



TranSparse [Ji et al., 2016] : method

- In previous work, no matter simple or complex, each relation has a transfer matrix with the same number of parameters...
- ... 💡 In TranSparse, the authors argue that **the complexity of a relation is proportional to the number of triplets linked by it**
 - The more data linked by the relation, the more knowledge it contains
 - Complex relations need more parameters to learn fully
 - Transfer matrices are replaced by **adaptive sparse matrices**, whose sparse degrees (i.e., non-zero entries learned by training = degrees of freedom) depend on the number of entities linked by relations
- Two versions
 - (1) **TranSparse (share)**. Entities use the same sparse matrix for projection. Sparsity depends on the relation complexity.
 - (2) **TranSparse (separate)**. Entities use different sparse matrices for projection. Sparsity depends on the amount of head/tail entities of the relation.
- Margin-based ranking loss

TranSparse [Ji et al., 2016] : share

Essentially TransR, but without over-parameterization. Every relation gets a projection matrix with a sparsity regulated by its complexity.

$$\theta_r = 1 - (1 - \theta_{min}) N_r / N_{r*}$$

Fraction of zero elements over the total number of elements
 Maximum number of linked entity pairs
 # of entity pairs linked by relation r
 hyperparameter $\in [0,1]$

$$\mathbf{M}_r(\theta_r) \in \mathbb{R}^{m \times n}$$

$$\mathbf{h}, \mathbf{t} \in \mathbb{R}^n, \quad \mathbf{h}_\perp, \mathbf{t}_\perp \in \mathbb{R}^m$$

Usually, $n = m \in \{20, 100\}$

$$\mathbf{h}_\perp = \mathbf{M}_r(\theta_r) \mathbf{h}, \quad \mathbf{t}_\perp = \mathbf{M}_r(\theta_r) \mathbf{t}$$

$$f_r(h, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_{l_1/l_2}$$

TranSparse [Ji et al., 2016] : separate

Essentially TransD (in the goal), but without over-parameterization. Every entity-relation pair gets a projection matrix with a sparsity regulated by its complexity. Better performance than *share* in most cases.

$$\theta_r^l = 1 - (1 - \theta_{min}) N_r^l / N_{r*}^{l*}$$

$l = h, t$
 Maximum number of entities linked at location l
 # of entities linked by relation r at location l

$$\mathbf{M}_r^h(\theta_r^h), \mathbf{M}_r^t(\theta_r^t) \in \mathbb{R}^{m \times n}$$

$$\mathbf{h}, \mathbf{t} \in \mathbb{R}^n, \quad \mathbf{h}_\perp, \mathbf{t}_\perp \in \mathbb{R}^m$$

$$\mathbf{h}_\perp = \mathbf{M}_r^h(\theta_r^h) \mathbf{h}, \quad \mathbf{t}_\perp = \mathbf{M}_r^t(\theta_r^t) \mathbf{t}$$

$$f_r(h, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_{l_1/l_2}$$

Many more

Model	Entity Embeddings	Relation Embeddings	Score Function
TransE	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$	$\mathbf{r} \in \mathbb{R}^d$	$-\ \mathbf{h} + \mathbf{r} - \mathbf{t}\ _{l_1/l_2}$
TransH	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$	$\mathbf{r} \in \mathbb{R}^d$	$-\ (\mathbf{h} - w_r^T \mathbf{h} w_r) + \mathbf{r} - (\mathbf{t} - w_r^T \mathbf{t} w_r)\ _2^2$
TransR	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$	$\mathbf{r} \in \mathbb{R}^k$	$-\ M_r \mathbf{h} + \mathbf{r} - M_r \mathbf{t}\ _2^2$
TransD	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$	$\mathbf{r} \in \mathbb{R}^k$	$-\ (w_r w_h^T + I) \mathbf{h} + \mathbf{r} - (w_r w_t^T + I) \mathbf{t}\ _2^2$
TransSparse	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$	$\mathbf{r} \in \mathbb{R}^k$	(sh) $-\ M_r(\theta_r) \mathbf{h} + \mathbf{r} - M_r(\theta_r) \mathbf{t}\ _2^2$ (sep) $-\ M_r^h(\theta_r^h) \mathbf{h} + \mathbf{r} - M_r^t(\theta_r^t) \mathbf{t}\ _2^2$
RotatE	$\mathbf{h}, \mathbf{t} \in \mathbb{C}^d$	$\mathbf{r} \in \mathbb{C}^d$	$-\ \mathbf{h} \circ \mathbf{r} - \mathbf{t}\ $
QuatE	$\mathbf{h}, \mathbf{t} \in \mathbb{H}^d$	$\mathbf{r} \in \mathbb{H}^d$	$-\left\ \mathbf{h} \otimes \frac{\mathbf{r}}{ \mathbf{r} } - \mathbf{t} \right\ $
HAKE	$\mathbf{h}_m, \mathbf{t}_m \in \mathbb{R}^d$ $\mathbf{h}_p, \mathbf{t}_p \in [0, 2\pi]^d$	$\mathbf{r}_m \in \mathbb{R}^d$ $\mathbf{r}_p \in [0, 2\pi]^d$	$-\ \mathbf{h}_m \circ \mathbf{r}_m - \mathbf{t}_m\ _2 - \lambda \left\ \sin \left((\mathbf{h}_p + \mathbf{r}_p - \mathbf{t}_p) / 2 \right) \right\ _1$
DualE	$\mathbf{h}, \mathbf{t} \in \mathbb{H}_{dual}^d$	$\mathbf{r} \in \mathbb{H}_{dual}^d$	$\langle \mathbf{h} \otimes \frac{\mathbf{r}}{ \mathbf{r} }, \mathbf{t} \rangle$
GIE	$\mathbf{h}, \mathbf{t} \in \mathbb{G}^d$	$\mathbf{r} \in \mathbb{G}^d$	$-(d_c(Inter(E_h, H_h, S_h), \mathbf{t}) + d_c(Inter(E_t, H_t, S_t), \mathbf{h})) + b$